

## APPENDIX A

### RECOMMENDATION REASONING INTERFACE

As shown in Table 14, we define a structured reasoning interface that specifies how a large language model should analyze mashup requirements and generate API recommendations. This interface encapsulates role assumptions, output constraints, and an explicit reasoning workflow, enabling the model to perform systematic analysis and decision-making rather than ad-hoc generation. In practical deployment, concrete mashup examples and recommendation scenarios are instantiated as inputs to this interface, allowing the model to ground its reasoning in realistic mashup creation contexts.

Tables 15-17 further illustrate the offline refinement process of the reasoning plan conducted by the Planning Agent, with a particular focus on the evolution of the reasoning workflow component. To improve readability, only representative iterations (the first, second, and fourth) are reported. Among them, the final iteration yields the best validation performance and is therefore adopted as the fixed reasoning interface used during online recommendation.

## APPENDIX B

### SEMANTIC ENRICHMENT INTERFACE

Table 18 and Table 19 specify the interface definitions of the Semantic Enrichment Agent for mashup and API enhancement, respectively. These interface specifications describe the expected input-output structure, alignment objectives, and normalization rules that govern how raw textual descriptions and categorical metadata are analyzed and transformed into semantically enriched representations.

The example sections illustrate representative outputs produced by the agent under this interface, demonstrating how the enrichment process enforces standardized expression, improves technical completeness, and maintains consistency across heterogeneous descriptions. In addition, Table 20 and Table 21 present illustrative mashup and API representations before and after semantic enrichment, thereby visualizing the transformation process and highlighting the practical effect of the proposed semantic alignment mechanism.

## APPENDIX C

### TIME COMPLEXITY ANALYSIS

This appendix analyzes the time complexity of the proposed MARS framework, with a focus on how the stage-wise design bounds computation as the repository scales.

The online recommendation pipeline consists of three major components: mashup-level candidate retrieval, lightweight API ranking, and large language model based collaborative recommendation under bounded input. Semantic enrichment over historical mashups and APIs is conducted offline and therefore does not contribute to online inference cost. During online recommendation, semantic enrichment is applied only once to the user-provided mashup requirement as a lightweight preprocessing step with bounded input length. Accordingly, the total online time complexity can be decomposed as

$$T = T_{\text{retrieval}} + T_{\text{rank}} + T_{\text{LLM}}. \quad (26)$$

### Mashup Candidate Retrieval

In the candidate filtering stage, MARS retrieves mashups that are semantically similar to the target requirement using a hybrid retrieval mechanism that combines dense vector similarity search and sparse inverted-index retrieval over the mashup corpus. The computational cost of this stage is dominated by similarity search and index lookup operations over the mashup collection. In the worst case, the retrieval cost scales linearly with the number of indexed mashups, yielding the following conservative upper bound:

$$T_{\text{retrieval}} = O(|\mathcal{M}|). \quad (27)$$

Although practical implementations rely on approximate nearest-neighbor search and efficient inverted indexes to achieve sublinear performance, the linear bound serves as a safe upper bound for complexity analysis.

### Candidate API Ranking

After mashup retrieval, only APIs that appear in the retrieved mashup set are considered. Let  $\mathcal{M}_{\text{final}}$  denote the bounded set of retrieved mashups, and let  $m$  denote the number of unique APIs extracted from these mashups after deduplication. Since the number of APIs per mashup is bounded and  $\mathcal{M}_{\text{final}}$  is fixed by design, API extraction and deduplication incur linear cost with respect to the number of extracted APIs.

The ranking stage computes frequency-based statistics and semantic similarity scores for each API in the reduced candidate set. This process requires a single pass over the  $m$  candidates, leading to the following time complexity:

$$T_{\text{rank}} = O(m). \quad (28)$$

In practice,  $m$  is typically much smaller than the size of the full API repository, which significantly reduces the computational burden of downstream processing.

### LLM-based Collaborative Recommendation

In the final stage, MARS generates recommendations using a fixed number of large language model based agents. Let  $M$  denote the number of agents involved in collaborative reasoning. Each agent performs a constant number of inference calls. The cost of a single inference call depends on the language model architecture and the length of the input sequence, where the input consists of the agent interface and a top- $K$  candidate set, and is therefore independent of the size of the API repository. Denoting the cost of a single inference as  $C_{\text{LLM}}$ , the time complexity of this stage is given by

$$T_{\text{LLM}} = O(M \cdot C_{\text{LLM}}). \quad (29)$$

Combining the above components, the overall online time complexity of MARS can be expressed as

$$T = O(|\mathcal{M}|) + O(m) + O(M \cdot C_{\text{LLM}}). \quad (30)$$

By progressively narrowing the candidate space and decoupling repository-scale operations from language model based reasoning, MARS avoids combinatorial exploration over the full API space. This design ensures that the most computationally expensive inference component operates under a fixed and controllable input size, enabling predictable computational cost and practical scalability in large-scale service recommendation environments.

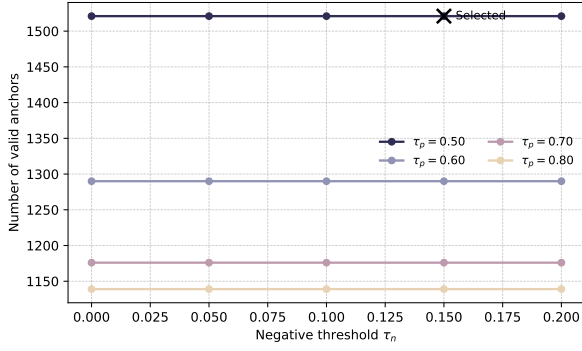


Fig. 7. Number of Valid Anchors under Different Threshold Settings

TABLE 5

Performance Comparison under Different Threshold Selections (Mean  $\pm$  Std, %)

|             | Recall@50        | NDCG@50          |
|-------------|------------------|------------------|
| (0.5, 0.15) | 80.32 $\pm$ 0.50 | 63.33 $\pm$ 1.24 |
| (0.5, 0.05) | 80.24 $\pm$ 0.31 | 63.28 $\pm$ 1.23 |
| (0.6, 0.15) | 80.31 $\pm$ 0.37 | 63.36 $\pm$ 1.37 |
| (0.7, 0.20) | 80.28 $\pm$ 0.45 | 63.27 $\pm$ 1.27 |

## APPENDIX D

### SENSITIVITY TO HYPERPARAMETER SETTINGS

#### Impact of Threshold Selection for Contrastive Pair Construction

We investigate the impact of threshold selection in contrastive pair construction by varying the positive and negative Jaccard overlap thresholds used to define mashup pairs. Specifically, we perform a grid search over  $\tau_p \in \{0.50, 0.60, 0.70, 0.80\}$  and  $\tau_n \in \{0.00, 0.05, 0.10, 0.15, 0.20\}$ , with the constraint  $\tau_p - \tau_n \geq 0.30$  to ensure sufficient API-overlap separation between positive and negative samples. For each threshold combination, we analyze the resulting training data in terms of the number of valid anchors (i.e., mashups that have at least one positive and one negative counterpart), the average number of positive and negative samples per anchor, and the average API overlap within each group.

The results are illustrated in Fig. 7. Our analysis shows that smaller positive thresholds generally lead to a larger number of valid anchors. In particular,  $\tau_p = 0.50$  consistently yields the highest anchor coverage, which helps preserve a broad and diverse set of training instances while maintaining semantic consistency among positive pairs. In contrast, increasing  $\tau_p$  reduces the number of eligible positive pairs and consequently limits the effective training set size, which may negatively affect training data coverage and sample diversity.

We further observe that the statistics of negative samples become saturated when  $\tau_n \leq 0.20$ . This phenomenon can be attributed to the fact that a large proportion of mashup pairs in the dataset exhibit little or no API overlap. As a result, lowering  $\tau_n$  below this level does not substantially change the composition or semantic characteristics of the negative sample pool after truncation, leading to highly similar distributions across different  $\tau_n$  settings.

TABLE 6

Grid Search for Selecting the Ranking Weight  $\lambda$  on the Validation Set in Retrieval Stage (Single Seed, %)

|     | Recall@50 | NDCG@50 |
|-----|-----------|---------|
| 0.0 | 85.39     | 69.11   |
| 0.1 | 84.70     | 68.81   |
| 0.2 | 84.70     | 68.97   |
| 0.3 | 84.70     | 68.98   |
| 0.4 | 84.70     | 69.23   |
| 0.5 | 84.70     | 69.46   |
| 0.6 | 84.72     | 69.72   |
| 0.7 | 84.56     | 69.64   |
| 0.8 | 84.06     | 67.22   |
| 0.9 | 83.17     | 53.17   |
| 1.0 | 78.59     | 30.88   |

TABLE 7

Performance Comparison under Different Ranking Hyperparameter  $\lambda$  (Mean  $\pm$  Std, %)

|     | Recall@10        | Precision@10     | F1@10            | NDCG@10          |
|-----|------------------|------------------|------------------|------------------|
| 0.0 | 67.43 $\pm$ 1.11 | 11.32 $\pm$ 0.12 | 19.38 $\pm$ 0.22 | 66.08 $\pm$ 0.44 |
| 0.3 | 67.20 $\pm$ 1.18 | 11.28 $\pm$ 0.12 | 19.32 $\pm$ 0.23 | 66.40 $\pm$ 0.86 |
| 0.5 | 67.86 $\pm$ 1.35 | 11.39 $\pm$ 0.16 | 19.51 $\pm$ 0.28 | 67.06 $\pm$ 0.94 |

Based on these observations, we adopt  $(\tau_p, \tau_n) = (0.50, 0.15)$  as the default configuration for contrastive data construction, as it provides a favorable balance between training data coverage and semantic separability. To further examine the sensitivity of the model to threshold selection, we additionally evaluate several representative threshold pairs during the retrieval stage. These configurations are chosen to reflect both more permissive settings (e.g., lower  $\tau_p$  or higher  $\tau_n$ , allowing more positive or harder negative pairs) and more restrictive settings (e.g., higher  $\tau_p$  or lower  $\tau_n$ , enforcing stricter composition-level similarity/dissimilarity).

Experimental results across multiple random seeds show only marginal variations in Recall@50 and NDCG@50 under different threshold settings, as reported in Table 5. This relative stability arises from the interaction between the threshold selection strategy and the underlying data characteristics. In particular, our strategy identifies a stable operating region in the parameter space in which the resulting contrastive datasets exhibit comparable scale, valid-anchor coverage, and supervision quality. Within this region, moderate expansions or contractions around the selected thresholds do not substantially alter the composition of positive and negative samples, leading to only minor fluctuations in retrieval performance. Overall, this insensitivity to small threshold variations indicating that the contrastive pair construction is robust to moderate threshold changes and reliably reflects composition-level API-overlap signals in mashup data.

#### Impact of the Ranking Weight $\lambda$

As defined in Eq. 17, the ranking score is computed as a weighted combination of the similarity-based score  $g(s, q)$  and the frequency-based score  $f(s; \mathcal{M}_{final})$ , where the hyperparameter  $\lambda$  controls their relative contributions. A larger  $\lambda$  emphasizes semantic alignment between an API and the target mashup requirement, while a smaller  $\lambda$  favors historically frequent APIs within the retrieved mashups.

TABLE 8

Performance Comparison of MARS-w/o-PR under Different Numbers of Recommender-Generated Options (Mean  $\pm$  Std, %)

| Setting | Recall@10        | Precision@10     | F1@10            | NDCG@10          |
|---------|------------------|------------------|------------------|------------------|
| N=1     | 64.08 $\pm$ 2.41 | 10.75 $\pm$ 0.34 | 18.42 $\pm$ 0.60 | 63.33 $\pm$ 1.65 |
| N=2     | 66.26 $\pm$ 1.71 | 11.12 $\pm$ 0.21 | 19.04 $\pm$ 0.39 | 65.29 $\pm$ 1.07 |
| N=3     | 66.48 $\pm$ 1.78 | 11.12 $\pm$ 0.17 | 19.04 $\pm$ 0.31 | 65.65 $\pm$ 1.27 |
| N=4     | 66.43 $\pm$ 1.82 | 11.15 $\pm$ 0.24 | 19.09 $\pm$ 0.43 | 65.48 $\pm$ 1.13 |

We first use the validation set to select an appropriate value of  $\lambda$ . Specifically, under a fixed random seed, we perform a grid search by sweeping  $\lambda$  from 0 to 1 and evaluate retrieval-stage performance. The results are summarized in Table 6. As  $\lambda$  increases (e.g., from 0.1 to 0.3), Recall@50 remains largely stable with a slight reduction compared to  $\lambda = 0$  while NDCG@50 improves modestly, suggesting that incorporating semantic relevance refines ranking quality with only minor loss in coverage. In the range  $\lambda \in [0.4, 0.6]$ , both metrics remain stable with steadily improved NDCG@50, reflecting a balanced contribution of popularity and semantic relevance. In contrast, when  $\lambda > 0.7$ , performance degrades noticeably (e.g., Recall@50 drops to 78.59 at  $\lambda = 1.0$  and NDCG@50 sharply declines to 30.88), suggesting that placing excessive weight on semantic similarity can lead to suboptimal ranking performance, potentially at the expense of frequency-based signals. Based on the validation results, we adopt  $\lambda = 0.5$  as the default setting.

To further assess the impact of  $\lambda$  beyond the single-seed validation selection, we conduct an additional sensitivity study on the test set. We choose three representative values  $\lambda \in \{0.0, 0.3, 0.5\}$ , corresponding to frequency-dominant, similarity-enhanced, and balanced configurations, and evaluate them across three different random seeds. The results are reported in Table 7. Overall, moderate fusion weights achieve a better trade-off:  $\lambda = 0.5$  yields the highest mean Recall@10 (67.86%) and NDCG@10 (67.06%), outperforming both the frequency-only setting ( $\lambda = 0$ ) and smaller fusion weights. These results indicate that selecting  $\lambda$  via validation-set grid search yields a stable and well-performing configuration, and that balancing historical co-usage signals with contextual semantic relevance is beneficial for effective API ranking in practice. The relatively modest performance differences across  $\lambda$  settings further suggest that semantic similarity primarily acts as a fine-grained ranking signal rather than a dominant factor in candidate selection, which is consistent with the ranking rationale described in Section 2.3.3.

#### Impact of Different Numbers of Recommender-Generated Options

Table 8 analyzes the impact of the number of Recommender-generated option candidates under the MARS-w/o-PR setting, where the explicit reasoning plan (i.e., chain-of-thought guidance) is removed to isolate the effect of collaborative proposal generation and validation-based selection. As shown in the table, increasing  $N$  from 1 to 2 yields consistent improvements across all evaluation metrics, with Recall@10 rising from 64.08% to 66.26% and NDCG@10 increasing from 63.33% to 65.29%. This suggests that relying on a single recommendation option may be

TABLE 9

Semantic Gap Before and After Semantic Enrichment (Mean  $\pm$  Std)

| Method   | Before              | After               |
|----------|---------------------|---------------------|
| Ours     | 0.7333 $\pm$ 0.0029 | 0.6333 $\pm$ 0.0051 |
| T2L2     | 0.7202 $\pm$ 0.0004 | 0.7191 $\pm$ 0.0031 |
| TextCNN  | 0.7202 $\pm$ 0.0004 | 0.7208 $\pm$ 0.0037 |
| ServeNet | 0.7202 $\pm$ 0.0004 | 0.7162 $\pm$ 0.0016 |
| MTFM     | 0.7189 $\pm$ 0.0029 | 0.7205 $\pm$ 0.0036 |
| MGSR     | 0.7226 $\pm$ 0.0010 | 0.7209 $\pm$ 0.0027 |
| NGCF     | 0.7219 $\pm$ 0.0003 | 0.7197 $\pm$ 0.0033 |
| SEHGN    | 0.7167 $\pm$ 0.0077 | 0.7168 $\pm$ 0.0077 |

TABLE 10

Average Cosine Similarity Before and After Semantic Rewriting (Mean  $\pm$  Std)

|                              | Before              | After               |
|------------------------------|---------------------|---------------------|
| Rewrite Categories           | 0.3972 $\pm$ 0.0001 | 0.5110 $\pm$ 0.0119 |
| Rewrite Tags                 | 0.3695 $\pm$ 0.0003 | 0.5100 $\pm$ 0.0010 |
| Rewrite Description (Mashup) | 0.3972 $\pm$ 0.0001 | 0.4263 $\pm$ 0.0179 |
| Rewrite Description (API)    | 0.3695 $\pm$ 0.0003 | 0.3882 $\pm$ 0.0054 |

insufficient to fully capture diverse aspects of complex mashup requirements, whereas introducing a small number of alternative proposals substantially improves functional coverage and ranking quality. When  $N$  is further increased to 3, the gains become marginal, and the performance largely saturates at  $N = 4$ , with only minor fluctuations in the metrics.

This trend is consistent with the effect of introducing controlled diversity through multiple recommendation options, which increases the likelihood of generating correct service compositions by covering different aspects of functional requirements. Meanwhile, the Reviewer Agent can reliably select high-quality options from the candidate set, making the overall collaborative framework less sensitive to imperfect individual recommendations.

It is important to note that this experiment is not used for hyperparameter selection. In our framework, the number of generated options  $N$  is chosen a priori based on empirical considerations and practical experience with LLM-based generation, rather than tuned on test data. Specifically, we adopt  $N = 3$  as a reasonable default to balance diversity, robustness, and computational overhead.

## APPENDIX E

### SUPPLEMENTARY ANALYSIS OF SEMANTIC ENRICHMENT

Table 9 reports the absolute Jensen–Shannon distance (JSD) values before and after semantic enrichment for our method and the baseline approaches

To address the concern that semantic enhancement may drift from the original user intent, we conduct two complementary semantic similarity analyses, under the assumption that conservative, intention-preserving enrichment should strengthen alignment with the original descriptions and categorical annotations. First, to evaluate whether rewriting categories/tags improves semantic alignment, we measure the cosine similarity between original mashup descriptions and (i) their original categories versus (ii) rewritten categories; for APIs, we analogously compare original API descriptions with original tags and rewritten tags. Second,

TABLE 11  
Performance Comparison of Different Configurations Related to API Ranking (Mean  $\pm$  Std, %)

|                | Recall@10        | Precision@10     | F1@10            | NDCG@10          |
|----------------|------------------|------------------|------------------|------------------|
| MARS           | 67.86 $\pm$ 1.35 | 11.39 $\pm$ 0.16 | 19.51 $\pm$ 0.28 | 67.06 $\pm$ 0.94 |
| Random-rank    | 65.81 $\pm$ 1.68 | 11.05 $\pm$ 0.21 | 18.92 $\pm$ 0.38 | 62.71 $\pm$ 1.41 |
| Retrieval-only | 66.06 $\pm$ 0.37 | 11.15 $\pm$ 0.40 | 19.09 $\pm$ 0.36 | 60.77 $\pm$ 0.71 |

to assess the effect of rewriting descriptions, we measure the cosine similarity between original categories/tags and (i) original descriptions versus (ii) rewritten mashup/API descriptions. As reported in Table 10, semantic enrichment consistently improves alignment across all settings. Rewriting categories increases the average cosine similarity for mashups from 0.3972 to 0.5110 and for APIs from 0.3695 to 0.5100, yielding absolute improvements of 0.1138 and 0.1405, respectively. When rewriting descriptions, the similarity also improves, albeit to a smaller extent, from 0.3972 to 0.4263 for mashups and from 0.3695 to 0.3882 for APIs. These findings are consistent with our hypothesis. Since categories and tags are derived from the original functional intent, increased alignment suggests that the rewritten representations remain anchored to the same semantic space rather than introducing unrelated or extraneous content. In other words, the enrichment process appears to reorganize and clarify existing information, rather than substantially altering the underlying intent.

## APPENDIX F SUPPLEMENTARY ANALYSIS OF THE API RANKING STRATEGY

Building on the previous analysis of ranking quality, Table 11 examines how different API ranking configurations affect the final recommendation performance of the agent, which are defined as follows:

- **Random-rank:** Randomly shuffles the top-50 APIs retrieved by MARS before.
- **Retrieval-only:** Directly uses the retriever’s top-10 APIs as recommendations, without re-ranking or agent-based reasoning.

The results show that randomly shuffling the top-50 candidate APIs leads to a clear degradation in recommendation quality. Compared with MARS, the Random-rank setting reduces Recall@10 from 67.86% to 65.81% and NDCG@10 from 67.06% to 62.71%, with the drop in NDCG being particularly pronounced. This indicates that the ordering of candidate APIs provides important signals for downstream scoring and selection, and that disrupting this order degrades the quality of relevance estimation and coverage assessment.

Despite this strong dependence on ordering, MARS improves Recall@10 from 66.06% to 67.86% and yields a substantial gain in NDCG@10 (from 60.77% to 67.06%) compared with the Retrieval-only baseline. These results suggest that the overall recommendation framework can identify relevant APIs beyond those favored by traditional frequency-based or semantic similarity-based ranking criteria, benefiting from the combination of structured retrieval, ranking, and validation-based selection.

TABLE 12  
Comparison with Different LLM Backbones (Mean  $\pm$  Std, %) **MARS** and **Native RAG** are implemented with **Qwen2.5-14B-Instruct**. Other baselines use their default large language models under the Native RAG setting.

|                                           | Recall@10        | Precision@10     | F1@10            | NDCG@10          |
|-------------------------------------------|------------------|------------------|------------------|------------------|
| <b>Small Model (Qwen2.5-14B-Instruct)</b> |                  |                  |                  |                  |
| MARS                                      | 67.86 $\pm$ 1.35 | 11.39 $\pm$ 0.16 | 19.51 $\pm$ 0.28 | 67.06 $\pm$ 0.94 |
| Native RAG                                | 60.18 $\pm$ 0.12 | 10.10 $\pm$ 0.08 | 17.30 $\pm$ 0.12 | 61.22 $\pm$ 0.11 |
| <b>Large Models (Native RAG)</b>          |                  |                  |                  |                  |
| DeepSeek-V3.1                             | 62.06 $\pm$ 5.31 | 10.42 $\pm$ 0.82 | 17.83 $\pm$ 1.42 | 59.26 $\pm$ 5.20 |
| Qwen3-plus                                | 66.56 $\pm$ 0.86 | 11.17 $\pm$ 0.07 | 19.13 $\pm$ 0.14 | 64.00 $\pm$ 1.00 |
| Gemini2.5-flash                           | 66.66 $\pm$ 1.26 | 11.19 $\pm$ 0.14 | 19.16 $\pm$ 0.25 | 64.90 $\pm$ 1.12 |
| ChatGPT-4o-mini                           | 66.22 $\pm$ 1.36 | 11.11 $\pm$ 0.16 | 19.04 $\pm$ 0.29 | 63.84 $\pm$ 1.16 |

TABLE 13  
Efficiency and Deployment Comparison between SotA LLM-based RAG and MARS. The local footprint indicates the minimum GPU memory required for local FP16 inference.

| Method     | Deployment            | Local Footprint  | Avg Tokens | Backbone    |
|------------|-----------------------|------------------|------------|-------------|
| Native RAG | Proprietary API       | N/A (Cloud-only) | 1,450      | SotA LLM    |
| MARS       | Private (Open-weight) | $\sim$ 28GB      | 19,664     | Qwen2.5-14B |

## APPENDIX G SUPPLEMENTARY ANALYSIS OF PRIVATE DEPLOYMENT

To evaluate whether MARS can serve as a practical alternative to LLM-based RAG systems in private deployment scenarios, we compare it with Native RAG instantiated using different large language model backbones. As shown in Table 12, MARS is implemented with Qwen2.5-14B-Instruct, a fully deployable open-weight model, yet achieves performance that remains competitive with Native RAG equipped with larger cloud-based proprietary LLMs. Specifically, MARS attains a Recall@10 of 67.86%, which is comparable to Native RAG using DeepSeek-V3.1 (62.06%), Qwen3-plus (66.56%), Gemini-2.5-flash (66.66%), and ChatGPT-4o-mini (66.22%). Similar trends are observed in Precision@10 and NDCG@10. These results indicate that MARS is suitable for private, data-sensitive, or resource-constrained deployment environments where direct reliance on state-of-the-art cloud-based models may be undesirable.

To further examine practical deployment considerations, we provide an empirical efficiency and deployment comparison in Table 13. As shown in Table 13, MARS incurs higher token consumption and runtime due to its explicit structured reasoning and multi-agent validation stages. However, these additional steps also strengthen the robustness of the framework and improve decision transparency by making the reasoning process more explicit and verifiable. The key distinction lies in deployment independence: Native RAG depends on proprietary cloud-based LLM APIs, whereas MARS operates with a fully deployable open-weight model. Taken together, these results suggest that MARS offers a viable and controllable alternative to proprietary LLM-based RAG systems, achieving competitive performance while enabling private deployment.

TABLE 14  
Recommender Agent Interface

### Background Explanation

Service recommendation refers to analyzing and recommending appropriate services or functionalities based on user needs or a specific application scenario. These services are typically provided in the form of APIs (Application Programming Interfaces). Your task, as a Mashup developer, is to analyze the Mashup's description, identify its functional requirements, and select a combination of APIs from the provided related\_apis. **Mashup**

### Examples

A Real Mashup From the Training Set

### Task

To ensure accurate recommendations, follow these steps to think step by step:

- Analyze Functional Requirements: Carefully read the Mashup description and extract the key functional needs.
- Filter APIs: From related\_apis, first select APIs that fully satisfy all functional requirements. If fewer than  $\{\{k\}\}$  such APIs exist, then include APIs that satisfy at least one functional requirement.
- Select  $\{\{k\}\}$  APIs: The output list must contain exactly  $\{\{k\}\}$  APIs. If there are fewer than  $\{\{k\}\}$  fully satisfying APIs, fill the remaining slots with APIs that partially satisfy the requirements but are still functionally relevant.
- Ensure Coverage: Verify that the final selected APIs, when combined, comprehensively cover all functional requirements of the Mashup. Adjust the selection if necessary to maintain full coverage.
- Retrieve Titles: For each selected API, extract only its title field and include it in the final output.

### Output Format

- The output should be a valid JSON object containing two keys: mashup and related\_apis.
- The value of mashup must exactly match the description provided in the input.
- The related\_apis key must contain a list of exactly  $\{\{k\}\}$  items, each representing the title of a selected API. No empty strings or placeholders.
- Only select titles from the provided related\_apis list. Do not create or modify any titles.
- Do not arbitrarily modify or shorten the content of the title name, and absolutely do not fabricate or invent any content.
- Double-check that all elements in the related\_apis list of your output are valid title entries from the input

### Output Requirements

- The JSON output must strictly include only the mashup and related\_apis fields.
- The related\_apis list must always have exactly  $\{\{k\}\}$  items. Selecting APIs only from the provided list of related\_apis.
- Avoid any additional comments or unnecessary text in the output.

### Task Input and Output Structure

Expected Input: {

```
  "mashup": "Description of the Mashup's functional requirements",
  "related_apis": [
    {
      "title": "API Title 1",
      "tags": ["Tag1", "Tag2"],
      "description": "API functionality description"
    },
    {
      "title": "API Title 2",
      "tags": ["Tag3", "Tag4"],
      "description": "API functionality description"
    },
    ...
    {
      "title": "API Title 20",
      "tags": ["Tag5", "Tag6"],
      "description": "API functionality description"
    }
  ]
}
```

Expected Output:

```
{
  "mashup": "Description of the Mashup's functional requirements",
  "related_apis": [
    "API Title 1",
    "API Title 2",
    "API Title 8",
    "API Title 9",
    "API Title 12",
    "API Title 18",
    "API Title 15",
    "API Title 10",
    "API Title 5",
    "API Title 4"
  ]
}
```

### Example

A Real Input-Output Pair Generated From the Training Set

TABLE 15  
Reasoning Plan at the First Iteration

- 
- 1) **Analyze Functional Requirements:** Carefully read the Mashup description and extract the core functional needs, focusing on the primary purpose, key features, and intended user interactions. Identify essential capabilities such as communication, data retrieval, mapping, integration with external services, or automation.
  - 2) **Filter APIs by Functional Relevance:** From the `related_apis` list, first identify APIs that directly enable the core functionality described in the mashup. Prioritize APIs that support the main use case—such as messaging, telephony, geolocation, content aggregation, or real-time data processing—over those that provide secondary or tangential capabilities like advertising, analytics, or general utility functions.
  - 3) **Select  $\{\{k\}\}$  APIs:** The output must contain exactly  $\{\{k\}\}$  APIs. Prefer APIs that are central to implementing the mashup’s primary function. If fewer than  $\{\{k\}\}$  APIs fully satisfy the core requirements, supplement with APIs that support auxiliary but relevant functionalities, ensuring all additions maintain clear functional alignment with the mashup’s goals.
  - 4) **Ensure End-to-End Coverage:** Verify that the selected set of APIs collectively enables all critical aspects of the mashup. Adjust the selection if necessary so that no essential function is left unsupported, and avoid over-representation of similar capabilities unless explicitly justified by the use case.
  - 5) **Retrieve Titles:** For each selected API, extract only its title field and include it in the final output. Ensure titles are returned in order of relevance, with the most functionally critical APIs listed first.
- 

TABLE 16  
Reasoning Plan at the Second Iteration

- 
- 1) **Analyze Functional Requirements:** Carefully read the Mashup description and extract the core functional needs, focusing on the primary purpose, key features, and intended user interactions. Identify essential capabilities such as communication, data retrieval, mapping, integration with external services, or automation. Distinguish between central functionalities that define the mashup’s core value and secondary features that support peripheral use cases.
  - 2) **Filter APIs by Functional Relevance:** From the `related_apis` list, identify APIs that directly enable the core functionality described in the mashup. Prioritize those that provide foundational support for the main use case—such as geolocation, real-time data access, content aggregation, messaging, or domain-specific data processing—over those offering generic utilities, broad infrastructure, or indirect enhancements like analytics, advertising, or platform hosting.
  - 3) **Select  $\{\{k\}\}$  APIs:** The output must contain exactly  $\{\{k\}\}$  APIs. Rank selections by how tightly they align with the mashup’s primary function, choosing first those without which the mashup could not exist in its intended form. If fewer than  $\{\{k\}\}$  APIs are strictly essential, supplement with APIs that enhance usability, enrich data, or support logical extensions of the core functionality, ensuring each addition contributes meaningfully to the overall user experience.
  - 4) **Ensure End-to-End Coverage:** Verify that the selected APIs collectively cover all critical aspects of the mashup, including data sourcing, presentation, interaction, and integration. Favor combinations that distribute responsibilities across distinct functional layers (e.g., one for data, one for visualization, one for user input) rather than overlapping redundant capabilities, unless redundancy is justified by the use case.
  - 5) **Retrieve Titles:** For each selected API, extract only its title field and include it in the final output. Return titles in descending order of functional importance—starting with the most indispensable API for the mashup’s operation—ensuring clarity and precision in representing the hierarchy of dependencies.
- 

TABLE 17  
Reasoning Plan at the Fourth Iteration (Best Performance)

- 
- 1) **Analyze Functional Requirements:** Carefully read the Mashup description and extract the core functional needs, focusing on the primary purpose, key features, and intended user interactions. Identify essential capabilities such as communication, data retrieval, mapping, integration with external services, or automation. Clearly distinguish between central functionalities that are indispensable to the mashup’s operation and secondary features that provide supplementary value.
  - 2) **Filter APIs by Direct Enablement:** From the `related_apis` list, identify APIs that directly enable the core functionality described in the mashup. Prioritize those whose services are necessary for the fundamental operation of the mashup—such as real-time messaging, geolocation, content creation, voice calling, or domain-specific data access—over those offering general-purpose tools, broad infrastructure support, or indirect enhancements like analytics, advertising, or generic social sharing.
  - 3) **Select  $\{\{k\}\}$  APIs:** The output must contain exactly  $\{\{k\}\}$  APIs. Rank selections by how critically they support the mashup’s primary function, choosing first those without which the mashup could not perform its intended task. If fewer than  $\{\{k\}\}$  APIs are strictly essential, supplement with APIs that meaningfully extend core functionality—such as enriching data, improving user experience, or enabling logical next-step interactions—ensuring each addition plays a distinct and justified role.
  - 4) **Ensure Functional Completeness:** Verify that the selected APIs collectively cover all critical aspects of the mashup, including input handling, data sourcing, processing logic, and output presentation. Favor combinations that span different functional layers (e.g., one for data ingestion, one for transformation, one for visualization) rather than clustering around similar capabilities, unless layered redundancy is explicitly required by the use case.
  - 5) **Retrieve Titles:** For each selected API, extract only its title field and include it in the final output. Return titles in descending order of functional importance—starting with the most mission-critical API—ensuring the sequence reflects the hierarchy of dependency and contribution to the mashup’s core operation.
-

TABLE 18  
Semantic Enrichment Agent Interface for Mashup Rewriting

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Background Instruction</div><div>You are a helpful AI assistant specializing in analyzing and improving mashup descriptions and categories. Your task is to enhance the clarity of mashup descriptions while preserving and potentially expanding their categories.</div><div>Input Format</div><div><pre>{<br/>  "title": "Mashup: [Name]",<br/>  "description": "Original description text",<br/>  "categories": ["category1", "category2"]<br/>}</pre></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <div><div>Instructions for Processing</div><div>DESCRIPTION ANALYSIS:</div><div><ul style="list-style-type: none"><li>• Analyze the existing description and enhance it to include:<ul style="list-style-type: none"><li>- Clear explanation of the mashup's primary functionality</li><li>- Key features and capabilities</li><li>- Target users or use cases</li><li>- Any geographical or technical limitations</li></ul></li><li>• Maintain concise yet comprehensive wording</li><li>• Preserve the original meaning while adding technical clarity</li><li>• Include any implicit technical requirements</li></ul></div><div>CATEGORY HANDLING:</div><div><ul style="list-style-type: none"><li>• Preserve all existing categories exactly as provided</li><li>• Suggest additional relevant categories based on:<ul style="list-style-type: none"><li>- Core functionality (e.g., search, integration, visualization)</li><li>- Technical components (e.g., APIs, data sources)</li><li>- Application domain (e.g., retail, social, finance)</li><li>- User interaction type (e.g., search, browse, analyze)</li></ul></li></ul></div></div>                                                                                                          |
| <div><div>Output Format</div><div><pre>{<br/>  "title": "Mashup: [Original Name]",<br/>  "enhanced_description": "Improved description with technical details and clarity",<br/>  "categories": ["origin", "and", "additional", "relevant", "categories"]<br/>}</pre></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <div><div>Example Usage</div><div>Input:</div><div><pre>{<br/>  "title": "Mashup: Krillion",<br/>  "description": "Shopping comparison service with local shopping focus. Search for national products and see availability at stores near you.",<br/>  "categories": ["Mapping", "eCommerce"]<br/>}</pre></div><div>Expected Output:</div><div><pre>{<br/>  "title": "Mashup: Krillion",<br/>  "enhanced_description": "A location-aware shopping comparison service that integrates national product databases with local store inventory systems. The mashup enables users to search for products across multiple retailers and view real-time availability in nearby physical stores. It combines geolocation services with retail inventory APIs to provide accurate, location-based shopping information and price comparisons.",<br/>  "categories": ["Mapping", "eCommerce", "Local Search", "Retail", "Geolocation", "Price Comparison", "Inventory Tracking"]<br/>}</pre></div></div>                                                                                                                                                                                                                                                              |
| <div><div>Additional Guidelines</div><div><ul style="list-style-type: none"><li>• Maintain Professional Tone:<ul style="list-style-type: none"><li>- Use clear, technical language</li><li>- Avoid marketing speak</li><li>- Be precise and factual</li></ul></li><li>• Description Enhancement Focus:<ul style="list-style-type: none"><li>- Technical integration points</li><li>- Core functionalities</li><li>- User interaction flows</li><li>- Data sources and APIs</li><li>- Geographical aspects (if relevant)</li><li>- Real-time vs static data handling</li></ul></li><li>• Category Suggestions Should:<ul style="list-style-type: none"><li>- Be relevant to the core functionality</li><li>- Reflect technical components</li><li>- Consider user interaction patterns</li><li>- Include industry-standard terminology</li><li>- Be specific but not overly narrow</li></ul></li><li>• Remember:<ul style="list-style-type: none"><li>- Never remove or modify existing categories</li><li>- Maintain the original title format</li><li>- Focus on technical accuracy and clarity</li><li>- Keep descriptions concise but complete</li><li>- Ensure all suggested categories are justified by the description</li></ul></li></ul></div></div> |

TABLE 19  
Semantic Enrichment Agent Interface for API Rewriting

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Background Instruction</div><div>You are an AI assistant specializing in clarifying API descriptions and suggesting relevant tags. Your task is to enhance API descriptions by focusing on their core functionality and features, and suggest additional relevant tags while preserving existing ones.</div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <div><div>Input Format</div><div><pre>{<br/>  "title": "API Name",<br/>  "description": "Original description text",<br/>  "tags": ["tag1", "tag2"]<br/>}</pre></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <div><div>Instructions for Processing</div><div><div>DESCRIPTION ENHANCEMENT:</div><div><ul style="list-style-type: none"><li>• Focus on the API's core functionality and features</li><li>• Specify supported platforms and interfaces</li><li>• Highlight key capabilities and use cases</li><li>• Describe integration methods</li><li>• Note special features (like localization)</li><li>• Keep technical accuracy</li><li>• Maintain concise writing</li></ul></div><div>TAG HANDLING:</div><div><ul style="list-style-type: none"><li>• Preserve all existing tags</li><li>• Suggest additional tags based on:<ul style="list-style-type: none"><li>- Core functionality</li><li>- Technical features</li><li>- Platforms supported</li><li>- Integration methods</li><li>- Usage domains</li></ul></li></ul></div></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <div><div>Output Format</div><div><pre>{<br/>  "title": "Origin API Name",<br/>  "enhanced_description": "Clear, structured description of the API",<br/>  "tags": ["original", "and", "additional", "relevant", "tags"]<br/>}</pre></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <div><div>Example Usage</div><div><div>Input:</div><div><pre>{<br/>  "title": "Google Maps API MASTER RECORD",<br/>  "description": "From their site: The Google Gadgets API provides a way to put third-party content onto the Google homepage. The main use of the Google Gadgets API is to create gadgets (mini-applications) that users can add to their Google personalized homepages and other Google properties. For example, you could create a photo album gadget, or a gadget that displays an interactive game. The API is designed to be simple enough so that anyone can turn his or her web page or application into a gadget.",<br/>  "tags": ["Mapping", "Viewer"]<br/>}</pre></div><div>Expected Output:</div><div><pre>{<br/>  "title": "Google Maps API MASTER RECORD",<br/>  "enhanced_description": "A comprehensive mapping service API that enables embedding of Google Maps functionality in web applications. Supports both JavaScript and Flash interfaces for seamless integration across web and mobile platforms. Features include extensive language support with 50+ localizations, region-specific mapping, and geocoding capabilities. Provides enterprise-grade functionality with secure HTTPS access and intranet integration options for Premier customers.",<br/>  "tags": ["Mapping", "Viewer", "Geolocation", "Mobile", "Enterprise", "Localization", "JavaScript", "Integration"]<br/>}</pre></div></div></div> |
| <div><div>Additional Guidelines</div><div><ul style="list-style-type: none"><li>• Never remove or modify existing categories</li><li>• Maintain the original title format</li><li>• Focus on technical accuracy and clarity</li><li>• Keep descriptions concise but complete</li><li>• Ensure all suggested categories are justified by the description</li></ul></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |



TABLE 20  
The Effect of Semantic Enrichment for Mashups

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mashup</b>               | Fone Findr                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Origin description</b>   | Calls you when you've lost your phone, and lets you record a public message telling everyone where you found it.                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Enhanced description</b> | A telephony-based service that utilizes GPS and cellular data to initiate calls to the user when their lost phone is located. It provides a message recording functionality for users to communicate with the person who found their phone, guiding them on how to return it safely. The service integrates location tracking and messaging capabilities to facilitate the recovery of misplaced mobile devices.                                                                                 |
| <b>Origin categories</b>    | ["Telephony", "Voice"]                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Enhanced categories</b>  | ["Lost and Found", "GPS Tracking", "Mobile Security", "Telephony", "Voice", "Messaging"]                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Mashup</b>               | Skype Social Network                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Origin description</b>   | Skype social network mashup.                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Enhanced description</b> | A social networking platform that integrates Skype's messaging and chat functionalities with mapping capabilities to create a seamless communication experience for users. This mashup allows users to connect with friends and family, engage in real-time conversations, and share their locations using interactive maps. It combines Skype's voice and text communication features with geolocation services to enhance social interactions and facilitate meetings in real-world locations. |
| <b>Origin categories</b>    | ["Social", "Mapping", "Messaging", "Chat"]                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Enhanced categories</b>  | ["Social", "Mapping", "Messaging", "Chat", "Geolocation", "Real-Time Communication", "Location Sharing"]                                                                                                                                                                                                                                                                                                                                                                                         |

TABLE 21  
The Effect of Semantic Enrichment for API

|                             |                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API</b>                  | Discogs API MASTER RECORD                                                                                                                                                                                                                                                                                                                              |
| <b>Origin description</b>   | From their site: Discogs is a user-built database containing information on artists, labels, and their recordings. Discogs is constantly growing as users submit releases to the database.                                                                                                                                                             |
| <b>Enhanced description</b> | An extensive, user-generated database API offering detailed information on artists, labels, and their recordings. It supports a wide range of data submissions from users, making it a dynamic and constantly evolving resource for music enthusiasts and professionals alike.                                                                         |
| <b>Origin tags</b>          | ["Music"]                                                                                                                                                                                                                                                                                                                                              |
| <b>Enhanced tags</b>        | ["Music", "Database", "User-Generated", "Dynamic", "Artist", "Label", "Recording"]                                                                                                                                                                                                                                                                     |
| <b>API</b>                  | Instapaper API MASTER RECORD                                                                                                                                                                                                                                                                                                                           |
| <b>Origin description</b>   | The Instapaper API allows third-party applications to add URLs to Instapaper. Instapaper is a service that lets users save long web pages to read at a later time. The API has a single method. It uses RESTful calls and responses are formatted in JSON.                                                                                             |
| <b>Enhanced description</b> | Instapaper's API enables the integration of their web-based bookmarking and reading service into third-party applications. With a straightforward RESTful interface, developers can programmatically save web pages for later reading, enhancing productivity and content management. Responses are JSON formatted, facilitating easy parsing and use. |
| <b>Origin tags</b>          | ["Bookmarks"]                                                                                                                                                                                                                                                                                                                                          |
| <b>Enhanced tags</b>        | ["Bookmarks", "Content Management", "RESTful", "Reading", "ProductivityWeb", "Services"]                                                                                                                                                                                                                                                               |